

Trajectory Optimization - Direct Methods

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 17, 2026

1. Overview: Direct Methods

Core Philosophy: "Discretize then Optimize"

Instead of solving infinite-dimensional differential equations (HJB, PMP), we transform the Optimal Control Problem (OCP) into a finite-dimensional **Nonlinear Programming (NLP)** problem.

The Process:

- 1 **Discretize** the time horizon $[0, T]$ into N intervals.
- 2 **Parameterize** state $x(t)$ and control $u(t)$ trajectories (e.g., piecewise constant, polynomials).
- 3 **Enforce** constraints (Dynamics, Limits) at discrete points.
- 4 **Solve** using off-the-shelf NLP solvers (IPOPT, SNOPT).

2. Family of Direct Methods

How do we handle the system dynamics $\dot{x} = f(x, u)$?

- **1. Single Shooting (Sequential):** Only discretize control u . Integrate dynamics to find x .
- **2. Multiple Shooting (Hybrid):** Discretize u and states at intervals. Integrate independently and enforce continuity.
- **3. Collocation (Simultaneous):** Discretize both x and u . Enforce dynamics as algebraic constraints at every step.

3. Single Shooting: Formulation

Decision Variables: $Y = [u_0, u_1, \dots, u_{N-1}]^T$

Algorithm:

- 1 Guess controls Y .
- 2 Integrate dynamics $\dot{x} = f(x, u)$ from x_0 to compute state trajectory $x(t; Y)$.
- 3 Evaluate Cost Function and Constraints based on the integrated trajectory.

NLP Formulation:

$$\begin{aligned} \min_Y \quad & \sum_{i=0}^{N-1} l(x_i(Y), u_i) + l_f(x_N(Y)) \\ \text{s.t.} \quad & g(x_i(Y), u_i) \leq 0 \end{aligned}$$

Note: Dynamics are satisfied by construction (Integration).

4. Numerical Integration: Error Analysis (Blackboard)

How accurate is our discretization? Let's analyze the **Explicit Euler** method.

$$x_{k+1} = x_k + hf(x_k, t_k)$$

Local Error Derivation (Taylor Series): True solution expansion:

$$x(t+h) = x(t) + h\dot{x}(t) + \frac{h^2}{2}\ddot{x}(t) + O(h^3)$$

Euler approximation:

$$\hat{x}(t+h) = x(t) + hf(x(t)) = x(t) + h\dot{x}(t)$$

Local Error:

$$e = x(t+h) - \hat{x}(t+h) = \frac{h^2}{2}\ddot{x}(t) + O(h^3) \approx O(h^2)$$

Since global error accumulates over $1/h$ steps, Global Error $\approx O(h)$. (First-Order Method)

5. Computing Sensitivities (Gradients)

To solve the NLP, we need gradients of the cost/constraints w.r.t decision variables u . Since x depends on u via integration, we need **ODEs Sensitivities**.

Chain Rule:

$$\frac{dx_{i+1}}{du} = \frac{\partial \Phi}{\partial x_i} \frac{dx_i}{du} + \frac{\partial \Phi}{\partial u}$$

where $\Phi(x_i, u_i)$ is the integrator function (e.g., Euler step).

Initialization: $\frac{dx_0}{du} = 0$ (Initial state is fixed).

6. Sensitivities: Explicit Euler (Blackboard)

For Explicit Euler: $\Phi(x, u) = x + hf(x, u)$

State Sensitivity Matrix (A_i):

$$A_i \triangleq \frac{\partial \Phi}{\partial x} = I + h \frac{\partial f}{\partial x}$$

Control Sensitivity Matrix (B_i):

$$B_i \triangleq \frac{\partial \Phi}{\partial u} = h \frac{\partial f}{\partial u}$$

Propagation:

$$\frac{dx_{i+1}}{du_j} = A_i \frac{dx_i}{du_j} + B_i \delta_{ij}$$

7. Sensitivities: Runge-Kutta 4 (Blackboard)

RK4 is more complex. Let's derive the chain rule for the stages k .

$$x_{i+1} = x_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Sensitivity w.r.t x_i (Recursive Chain Rule):

$$\frac{\partial k_1}{\partial x_i} = f_x(x_i)$$

$$\frac{\partial k_2}{\partial x_i} = f_x\left(x_i + \frac{h}{2}k_1\right) \left(1 + \frac{h}{2} \frac{\partial k_1}{\partial x_i}\right)$$

$$\frac{\partial k_3}{\partial x_i} = f_x\left(x_i + \frac{h}{2}k_2\right) \left(1 + \frac{h}{2} \frac{\partial k_2}{\partial x_i}\right)$$

...

This must be propagated for every step!

8. Multiple Shooting: Formulation

Motivation: Single shooting is unstable for unstable systems (errors blow up).

Idea: Break the trajectory into segments and integrate independently.

Decision Variables: $Y = [s_0, u_0, s_1, u_1, \dots, s_N]^T$

- s_i : State at the beginning of interval i (Optimization variable).
- u_i : Control input for interval i .

Constraints (Defects): We must enforce continuity between segments:

$$s_{i+1} - \Phi(s_i, u_i) = 0 \quad \forall i = 0 \dots N-1$$

This adds equality constraints to the NLP but improves stability significantly.

9. Direct Collocation: Formulation

Idea: Approximate state trajectory $x(t)$ with polynomials (e.g., Splines) and enforce dynamics as algebraic constraints. No explicit integration!

Implicit Midpoint Rule (Constraint):

$$\frac{s_{i+1} - s_i}{h} - f\left(\frac{s_i + s_{i+1}}{2}, u_i\right) = 0$$

Features:

- **Very Sparse NLP:** Dependencies are local (only i and $i + 1$).
- **Fast:** Hessians and Jacobians are cheap to compute.
- **Accuracy:** Depends on the polynomial order (e.g., Hermite-Simpson is $O(h^4)$).

10. Sparsity Patterns (NLP Structure)

Visualizing the Jacobian of Constraints:

Single Shooting: Dense Lower Triangular (Sensitivity propagation couples everything).

$$\begin{bmatrix} * & 0 & 0 \\ * & * & 0 \\ * & * & * \end{bmatrix}$$

Collocation / Multiple Shooting: Block Diagonal / Banded Structure (Local dependencies).

$$\begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & 0 \\ 0 & 0 & * & * \end{bmatrix}$$

Sparse solvers (e.g., SNOPT) exploit this for speed.

11. Summary: Method Comparison

	Single Shooting	Multiple Shooting	Collocation
Variables	u only	u, x (Coarse)	u, x (Fine)
Dynamics	Integrated (ODE)	Integrated (ODE)	Constraint (Algebraic)
NLP Size	Small (Dense)	Medium (Sparse)	Large (Very Sparse)
Stability	Poor (Blow-up)	Good	Good
Use Case	Stable Systems	General Robotics	Complex Trajectories

Conclusion: For robotics, **Multiple Shooting** or **Collocation** is preferred due to stability and sparsity exploitation.